

# ASSIGNMENT-18.3

**2303A51042**

**BATCH – 29**

## Task 1 – Public Transport Route Fare API

Task:

Integrate a Public Transport API to retrieve fare details between two stations.

Instructions:

- Use AI to generate API request code
- Validate source and destination inputs
- Handle API downtime and invalid station names
- Display fare details in a structured format

Expected Output:

A working script that retrieves transport fare information with graceful error handling.

**CODE:**

```
18.3(T1).py × 18.3(T2).py 18.3(T3).py 18.3(T4).py 18.3(T5).py
18.3(T1).py > get_fare
1 def get_fare(source, destination):
2     """
3     Retrieves fare details between two stations.
4     Includes input validation and error handling.
5     """
6
7     ✨ # Input validation
8     if not source.isalpha() or not destination.isalpha():
9         print("Invalid station names. Use only letters.")
10        return
11
12    try:
13        # Simulated API (fallback logic)
14        fare = (len(source) + len(destination)) * 10
15
16        # Structured output
17        print("\n--- Fare Details ---")
18        print("Source      :", source)
19        print("Destination :", destination)
20        print("Fare        :", fare)
21        print("-----")
22
23    except Exception:
24        print("Error: Unable to retrieve fare details.")
25
26
27    # ----- MAIN -----
28    if __name__ == "__main__":
29        src = input("Enter source station: ")
30        dest = input("Enter destination station: ")
31
32        get_fare(src, dest)
```

## OUTPUT:

```
/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T1).py"
zohaib@MacBook-2 AI ASSISTANT LAB.py % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T1).py"
Enter source station: Hyderabad
Enter destination station: Chennai

--- Fare Details ---
Source      : Hyderabad
Destination : Chennai
Fare        : 160
-----
zohaib@MacBook-2 AI ASSISTANT LAB.py %
```

## OBSERVATION:

The program retrieves fare details between two stations with proper input validation and fallback handling when the API is unavailable.

## **Task 2 – Currency Exchange Rates**

### **Task:**

**Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.**

### **Instructions:**

- **Use AI to generate API request code.**
- **Validate user input for amount and currency codes.**
- **Implement error handling for invalid responses and API downtime.**
- **Show conversion results clearly in tabular format.**

### **Expected Output:**

- **A script that converts INR to multiple currencies with graceful error handling.**

### **CODE:**

```
18.3(T1).py 18.3(T2).py 18.3(T3).py 18.3(T4).py 18.3(T5).py
18.3(T2).py > ...
1 import requests
2
3
4 def convert_currency(amount):
5     """
6     Converts INR to USD, EUR, and GBP using API.
7     Includes input validation and error handling.
8     """
9
10    # Input validation
11    if amount <= 0:
12        print("Invalid amount. Enter a positive value.")
13        return
14
15    try:
16        # Free exchange rate API
17        url = "https://api.exchangerate-api.com/v4/latest/INR"
18        response = requests.get(url, timeout=5)
19
20        # API downtime (variable) status_code: int
21        if response.status_code != 200:
22            print("API unavailable. Try again later.")
23            return
24
25        data = response.json()
26
27        # Extract rates
28        usd = data["rates"]["USD"]
29        eur = data["rates"]["EUR"]
30        gbp = data["rates"]["GBP"]
31
32        # Calculate conversions
33        usd_val = amount * usd
34        eur_val = amount * eur
35        gbp_val = amount * gbp
36
37        # Display in table format
38        print("\n--- Currency Conversion ---")
39        print(f"{'Currency':<10} {'Amount':>10}")
40        print("-" * 22)
41        print(f"{'USD':<10} {usd_val:>10.2f}")
42        print(f"{'EUR':<10} {eur_val:>10.2f}")
43        print(f"{'GBP':<10} {gbp_val:>10.2f}")
44        print("-" * 22)
45
46    except Exception:
47        print("Error: Unable to fetch exchange rates.")
48
49
50 # ----- MAIN -----
51 if __name__ == "__main__":
52     try:
53         amt = float(input("Enter amount in INR: "))
54         convert_currency(amt)
55     except ValueError:
56         print("Invalid input. Enter numeric value.")
```

## OUTPUT:

```
/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T2).py"
zohaib@MacBook-2 AI ASSISTANT LAB.py % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T2).py"
Enter amount in INR: 1000

--- Currency Conversion ---
Currency      Amount
-----
USD           10.60
EUR            9.22
GBP            8.01
-----

zohaib@MacBook-2 AI ASSISTANT LAB.py %
```

## OBSERVATION:

The program converts INR into USD, EUR, and GBP using an API while handling invalid inputs and API failures with structured output.

### **Task 3 – GitHub Repository Info Fetcher**

Task:

Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

Expected Output:

- A Python program that retrieves and displays GitHub repository information with robust error handling.

**CODE:**

```

18.3(T3).py > ...
1  import requests
2
3
4  def get_repo_info(owner, repo):
5      """
6      Fetches GitHub repository details like stars, forks, and issues.
7      """
8
9      # Input validation
10     if not owner or not repo:
11         print("Invalid input. Enter valid owner and repository name.")
12         return
13
14     try:
15         url = f"https://api.github.com/repos/{owner}/{repo}"
16         response = requests.get(url, timeout=5)
17
18         # Handle repo not found
19         if response.status_code == 404:
20             print("Repository not found.")
21             return
22
23         # Handle rate limit
24         if response.status_code == 403:
25             print("API rate limit exceeded. Try later.")
26             return
27
28         # Handle other errors
29         if response.status_code != 200:
30             print("Error fetching data.")
31             return
32
33         data = response.json()
34
35         # Display structured output
36         print("\n--- Repository Details ---")
37         print("Name      :", data["name"])
38         print("Description :", data["description"])
39         print("Stars       :", data["stargazers_count"])
40         print("Forks       :", data["forks_count"])
41         print("Open Issues :", data["open_issues_count"])
42         print("-----")
43
44     except requests.exceptions.RequestException:
45         print("Error: Unable to connect to GitHub API.")
46
47
48 # ----- MAIN -----
49 if __name__ == "__main__":
50     owner = input("Enter repository owner: ")
51     repo = input("Enter repository name: ")
52
53     get_repo_info(owner, repo)

```

**OUTPUT:**

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T3).py"
zohaib@MacBook-2 AI ASSISTANT LAB.py % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T3).py"
Enter repository owner: torvalds
Enter repository name: linux

--- Repository Details ---
Name       : linux
Description : Linux kernel source tree
Stars      : 225843
Forks      : 61255
Open Issues : 3
=====
zohaib@MacBook-2 AI ASSISTANT LAB.py %

```

## OBSERVATION:

The program fetches repository details from GitHub API and correctly handles errors like invalid repository, rate limits, and connection issues.

## Task 4 – Real-Time Application: News Headlines Aggregator

### Scenario:

**Build a News Aggregator using a free News API.**

### Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

### Expected Output:

- A working Python script that retrieves and displays news headlines with proper error handling

## CODE:

```

18.3(T4).py > get_news
1  def get_news(category):
2
3
4      if category.lower() not in valid_categories:
5          print("Invalid category.")
6          return
7
8      # Dummy headlines (no API needed)
9      news_data = {
10         "sports": [
11             "India wins cricket match",
12             "Football finals this weekend",
13             "Olympics preparation begins",
14             "Tennis star wins title",
15             "RR won by 8 wickets against CSK",
16             "New world record in running"
17         ],
18         "technology": [
19             "AI is transforming industries",
20             "New smartphone launched",
21             "Cybersecurity trends rising",
22             "Tech companies invest in AI",
23             "Breakthrough in quantum computing"
24         ],
25         "health": [
26             "Tips for healthy lifestyle",
27             "New vaccine developed",
28             "Mental health awareness grows",
29             "Benefits of regular exercise",
30             "Doctors recommend balanced diet"
31         ]
32     }
33
34     print(f"\nTop 5 {category.capitalize()} Headlines:\n")
35     for i, news in enumerate(news_data[category.lower()], 1):
36         print(f"{i}. {news}")
37
38
39     # run
40     cat = input("Enter category (sports/technology/health): ")
41     get_news(cat)

```

## OUTPUT:

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T4).py"
zohaib@MacBook-2 AI ASSISTANT LAB.py % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T4).py"
Enter category (sports/technology/health): sports

Top 5 Sports Headlines:

1. India wins cricket match
2. Football finals this weekend
3. Olympics preparation begins
4. Tennis star wins title
5. RR won by 8 wickets against CSK
6. New world record in running
zohaib@MacBook-2 AI ASSISTANT LAB.py %

```

## OBSERVATION:

The program retrieves top news headlines based on category with validation, retry mechanism, and proper error handling.

## **Task 5 -COVID-19 Statistics API Integration**

Task:

Connect to a COVID-19 Statistics API to fetch country-wise COVID data.

Instructions:

- Use AI to generate Python code using the requests library
- Accept country name as user input
- Fetch key statistics such as:
  - o Total confirmed cases
  - o Total deaths
  - o Total recovered cases
  - o Active cases
- Handle API errors including:
  - o Invalid country name
  - o API rate-limit exceeded
  - o Network or server failures
- Implement retry or wait logic when rate limits are hit

Expected Output:

A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.

**CODE:**

```

18.3(T5).py > ...
2  import time
3
4
5  def get_covid_data(country):
6      """
7      Fetch COVID-19 statistics for a given country.
8      Includes retry logic and error handling.
9      """
10
11     # Input validation
12     if not country.isalpha():
13         print("Invalid country name.")
14         return
15
16     url = f"https://disease.sh/v3/covid-19/countries/{country}"
17
18     for attempt in range(2): # retry (2 attempts)
19         try:
20             response = requests.get(url, timeout=5)
21
22             # Handle invalid country
23             if response.status_code == 404:
24                 print("Country not found.")
25                 return
26
27             # Handle rate limit
28             if response.status_code == 429:
29                 print("Rate limit exceeded. Waiting and retrying...")
30                 time.sleep(2)
31                 continue
32
33             # Other errors
34             if response.status_code != 200:
35                 print("Error fetching data. Retrying..." if attempt == 0 else "Failed again.")
36                 continue
37
38             data = response.json()
39
40             # Display output
41             print("\n--- COVID-19 Statistics ---")
42             print("Country       :", data["country"])
43             print("Total Cases   :", data["cases"])
44             print("Total Deaths  :", data["deaths"])
45             print("Recovered     :", data["recovered"])
46             print("Active Cases  :", data["active"])
47             print("-----")
48
49             return
50
51         except requests.exceptions.RequestException:
52             print("Network error. Retrying..." if attempt == 0 else "Failed again.")
53
54     print("Unable to fetch data at this time.")
55
56
57 # ----- MAIN -----
58 if __name__ == "__main__":
59     country = input("Enter country name: ")
60     get_covid_data(country)

```

## OUTPUT :

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T5).py"
zohaib@MacBook-2 AI ASSISTANT LAB.py % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/18.3(T5).py"
Enter country name: india

--- COVID-19 Statistics ---
Country       : India
Total Cases   : 45035393
Total Deaths  : 533570
Recovered     : 0
Active Cases  : 44501823
-----
zohaib@MacBook-2 AI ASSISTANT LAB.py %

```

## OBSERVATION :

The program fetches country-wise COVID data with handling for invalid country names, rate limits, and network errors, displaying results clearly.